

RTOS-Based Predictive Maintenance System for Electric Motor

Using ESP32, FreeRTOS, MQTT, and Blynk IoT

Divyanshu Kalal

Dept. of Electronics & Communication Engineering
23BEC053@nirmauni.ac.in

Happy Patel

Dept. of Electronics & Communication Engineering
23BEC062@nirmauni.ac.in

Abstract—Industrial electric motors are critical assets in manufacturing and automation environments. Unexpected motor failures result in costly production downtime, equipment damage, and safety hazards. This paper presents the design and hardware implementation of a real-time IoT-based predictive maintenance system for electric motors using the ESP32 microcontroller running FreeRTOS. The system monitors two key motor health parameters: temperature, measured via a 10 k Ω NTC thermistor on GPIO 34, and vibration, captured using an MPU6050 six-axis IMU interfaced over I²C on GPIO 21/22. The MPU6050 is driven through direct raw register reads and calibrated over 2000 samples at startup. Vibration is computed as the gravity-compensated acceleration deviation: $V = |\sqrt{a_x^2 + a_y^2 + a_z^2} - 1.0|$. Five concurrent FreeRTOS tasks — `TemperatureTask`, `MPUSensorTask`, `FaultTask`, `MQTTTask`, and `BlynkTask` — share data through volatile global variables. Sensor data is published to a Mosquitto MQTT broker on three topics (`motor/temp`, `motor/vibration`, `motor/status`), while the Blynk IoT mobile dashboard provides live visualisation and push-notification alerting via `logEvent()` on every fault-state transition. Hardware testing confirmed accurate sensor readings, reliable MQTT telemetry, and correct three-level fault classification (Normal/High Vibration/Overheat).

Index Terms—Predictive Maintenance, ESP32, FreeRTOS, MPU6050, NTC Thermistor, MQTT, Blynk IoT, Vibration Monitoring, Temperature Monitoring, Real-Time Embedded System, IoT.

I. INTRODUCTION

Electric motors account for approximately 45% of global electricity consumption and are deployed across manufacturing, transport, and building services [1]. Despite their robustness, motors are susceptible to progressive degradation from thermal stress, mechanical imbalance, bearing wear, and rotor eccentricity. Industry data indicates that bearing failures and winding insulation breakdowns cause over 60% of all motor failures, and that most of these faults develop gradually — producing measurable physical signatures well before catastrophic failure [2].

Predictive maintenance (PdM) is a condition-based strategy that monitors actual equipment health and schedules intervention only when measurements approach fault thresholds. Compared to time-based preventive maintenance or purely corrective approaches, PdM reduces unplanned downtime, extends equipment service life, and lowers total maintenance cost.

The convergence of affordable microcontrollers, MEMS sensors, and Wi-Fi-enabled IoT platforms has made deploying PdM nodes on individual machines economically viable even for small-scale operations. This project implements a complete, hardware-verified predictive maintenance system using:

- **ESP32 WROOM-32** — dual-core 240 MHz SoC with Wi-Fi, 12-bit ADC, and I²C.
- **NTC Thermistor (10 k Ω)** — motor surface temperature via GPIO 34.
- **MPU6050** — six-axis IMU for vibration measurement via I²C (0x68).
- **FreeRTOS** — five concurrent tasks for sensing, detection, and communication.
- **MQTT (Mosquitto)** — lightweight publish/subscribe telemetry over Wi-Fi.
- **Blynk IoT** — mobile dashboard with `logEvent()` push-notification alerting.

A. Objectives

- 1) Implement real-time temperature monitoring using the Steinhart–Hart equation on an NTC thermistor.
- 2) Implement real-time vibration monitoring via raw MPU6050 I²C reads with 2000-sample startup calibration.
- 3) Deploy five concurrent FreeRTOS tasks for deterministic, non-blocking operation.
- 4) Transmit motor health data to a Mosquitto MQTT broker on three dedicated topics.
- 5) Provide mobile monitoring and push-notification fault alerts via Blynk IoT.
- 6) Classify motor health into three states: Normal (0), High Vibration (1), Overheat (2).

II. LITERATURE SURVEY AND STATE OF THE ART

A. Motor Condition Monitoring

Early condition monitoring relied on periodic manual inspections with portable vibration analysers and infrared thermometers [2]. Permanent online monitoring systems appeared in the 2000s using dedicated signal-conditioning hardware and PLCs, but at costs that restricted adoption to high-value machinery. The emergence of low-cost MEMS sensors and Wi-

Fi-enabled SoCs subsequently opened the door to affordable, always-connected monitoring nodes.

B. IoT-Based Monitoring Nodes

Patel et al. [3] demonstrated an ESP8266-based Wi-Fi vibration monitoring node transmitting RMS acceleration to a cloud database at 1 Hz. However, their bare-metal superloop implementation caused non-deterministic sampling under network load, with occasional data loss during TCP reconnection. Kumar and Singh [4] proposed a Raspberry Pi-based system using three-phase current signature analysis and SVM classification achieving 94.3% accuracy, but required expensive current probes and substantial processing hardware — limiting field applicability.

C. FreeRTOS in Embedded Monitoring

FreeRTOS provides preemptive scheduling, inter-task primitives, and a memory footprint suitable for microcontrollers with as little as 8 kB RAM [7]. Sharma et al. [8] demonstrated a 78% improvement in sensor sampling regularity on an STM32 FreeRTOS node compared to a superloop baseline. The `vTaskDelay()`-based periodic task pattern — used throughout this project — was identified as the most reliable scheduling approach for sensor acquisition. The use of `volatile` global variables for inter-task scalar data sharing is a well-established lightweight pattern on 32-bit architectures where single-word reads and writes are inherently atomic [7].

D. MQTT Protocol

MQTT (ISO/IEC 20922) uses a publish-subscribe model with minimal packet overhead and three QoS delivery guarantees [5]. Light et al. [6] showed MQTT consumed up to 93% less bandwidth than HTTP for equivalent telemetry payloads, making it well-suited for Wi-Fi-connected embedded nodes. Mosquitto, used in this project, is a fully OASIS-compliant open-source broker supporting thousands of simultaneous client connections.

E. Blynk IoT Platform

Blynk 2.0 maps microcontroller virtual pins to mobile app widgets without requiring native app development [9]. The `logEvent()` API fires named push notifications on any platform event — a pull-free alerting mechanism rarely available in open-source low-cost monitoring solutions.

F. MPU6050 in Vibration Sensing

At its default ± 2 g full-scale range, the MPU6050 provides 16384 LSB/g resolution [11]. The standard approach for vibration characterisation is to compute the Euclidean norm of the three acceleration axes and subtract the gravitational 1 g component to isolate dynamic vibration — exactly the method used in this work. Startup offset calibration over a statistically significant sample count is critical, particularly for the Z-axis where gravity must also be removed.

G. Research Gap

A unified implementation combining raw-register MPU6050 vibration sensing, NTC temperature monitoring, five-task FreeRTOS multitasking, MQTT telemetry, and Blynk push-notification alerting within a single ESP32 Arduino firmware — verified on physical hardware — has not been fully documented in the reviewed literature. This project fills that gap.

III. LIMITATIONS OF CURRENTLY AVAILABLE TECHNOLOGY

A. High Cost of Commercial Systems

Enterprise-grade platforms such as SKF @ptitude, Emerson AMS Suite, and National Instruments InsightCM range from \$10,000 to \$50,000 for a complete multi-channel installation. This cost structure is prohibitive for SMEs, research laboratories, and educational institutions where the case for condition monitoring is equally valid but budgets are severely constrained.

B. Wired Infrastructure Requirements

Legacy systems use Modbus RTU, PROFIBUS, or 4–20 mA analogue loops, requiring dedicated shielded cabling from each sensor to a central PLC. Installing fixed wiring in operational plants is disruptive, expensive, and physically impractical on rotating equipment. Wireless industrial alternatives (ISA100.11a, WirelessHART) remain costly and require dedicated network infrastructure.

C. No Real-Time Remote Monitoring

Conventional data loggers store readings locally. Maintenance personnel must physically retrieve and analyse data offline, introducing detection delays for developing faults. Systems offering remote access typically use proprietary closed-source dashboards with no mobile-native alerting.

D. Bare-Metal Firmware Limitations

Single-threaded superloop firmware — common in Arduino/ESP8266-based systems — causes non-deterministic sampling during network operations (Wi-Fi reconnection, TCP handshakes), missed sensor samples during MQTT reconnection, and an inability to maintain separate configurable update rates for different sensors (e.g., 1 Hz temperature vs. 5 Hz vibration). These issues are directly addressed in this project via FreeRTOS multitasking.

E. Binary Fault Classification

Most low-cost implementations offer only binary alarms (fault/no-fault) based on a single threshold. This produces high false-positive rates under transient conditions and cannot distinguish fault types. A multi-level status (Normal/Warning/Critical) provides more actionable information.

F. No Push-Notification Alerting

Dashboard-based systems require active user polling — the operator must open the app to check status. Proactive push notifications on state transitions, as provided by Blynk `logEvent()`, are rarely found in open-source low-cost solutions.

TABLE I: Comparison of Existing vs. Proposed System

Parameter	Existing	Proposed
Hardware Cost	\$1k–\$50k	<\$15
RTOS Tasks	Rare	5 FreeRTOS
Connectivity	Wired/Proprietary	Wi-Fi + MQTT
Push Alerts	No	Blynk <code>logEvent</code>
Fault Levels	Binary	3-level (0/1/2)
Sensor Rates	Fixed	1 Hz / 5 Hz
Open-Source	Vendor-locked	Fully open

IV. PROPOSED SOLUTION AND METHODOLOGY

A. System Overview

The proposed system is a four-layer IoT node built on the ESP32 running FreeRTOS. It acquires temperature and vibration data, classifies the motor health state, and simultaneously transmits telemetry to a Mosquitto MQTT broker and a Blynk IoT mobile dashboard — managed by five independent FreeRTOS tasks.

B. Hardware Components

TABLE II: Hardware Components

Component	Description
ESP32 WROOM-32	Dual-core 240 MHz; Wi-Fi; 12-bit ADC; I ² C
NTC Thermistor 10 kΩ	Voltage-divider on GPIO 34 (ADC1 CH6)
MPU6050 (0x68)	6-axis IMU; SDA=GPIO 21, SCL=GPIO 22
Resistor 10 kΩ	Series resistor in NTC divider
5 V USB Supply	Powers ESP32 dev board

C. Temperature Measurement

The NTC thermistor forms a voltage-divider with a fixed 10 kΩ resistor. The ADC count is clamped to [1, 4094] to prevent division-by-zero, then the thermistor resistance is computed:

$$R_{NTC} = R_{fixed} \times \frac{ADC}{ADC_{max} - ADC} \quad (1)$$

Temperature is derived using the Steinhart–Hart simplified equation:

$$T [^{\circ}\text{C}] = \frac{1}{\frac{1}{T_0} + \frac{1}{B} \ln\left(\frac{R_{NTC}}{R_0}\right)} - 273.15 \quad (2)$$

where $T_0 = 298.15$ K, $B = 3950$ K, and $R_0 = 10,000$ Ω.

D. Vibration Measurement and Calibration

The MPU6050 is initialised by writing 0x00 to Power Management register 0x6B. All sensor data is retrieved by writing register address 0x3B and requesting 14 bytes (ax, ay, az, temp, gx, gy, gz — 2 bytes each). Each axis value is reconstructed as:

$$ax_{raw} = (\text{highByte} \ll 8) | \text{lowByte} \quad (3)$$

Startup calibration averages 2000 stationary samples per axis. The Z-axis additionally removes the gravitational component:

$$az_{offset} = \frac{\sum_{i=1}^{2000} az_i}{2000} - 16384 \quad (4)$$

In `MPUSensorTask`, calibration offsets are subtracted before conversion to g -units ($\div 16384$), and vibration magnitude isolates dynamic motion:

$$V_{mag} = \left| \sqrt{a_x^2 + a_y^2 + a_z^2} - 1.0 \right| \quad (5)$$

A perfectly stationary sensor produces $V_{mag} \approx 0$.

E. FreeRTOS Task Architecture

Five tasks are created with `xTaskCreate()` at priority 1, each with a 4096-byte stack. Data is shared via three volatile global variables — safe for scalar reads/writes on the 32-bit ESP32 without additional synchronisation.

TABLE III: FreeRTOS Task Summary

Task	Period	Function
TemperatureTask	1000 ms	ADC read → Steinhart–Hart → <code>g_temp</code>
MPUSensorTask	200 ms	Raw I ² C read → calibrate → V_{mag} → <code>g_vibration</code>
FaultTask	1000 ms	Threshold check → <code>g_status</code> (0/1/2)
MQTTTask	2000 ms	Reconnect if needed → publish 3 topics
BlynkTask	1000 ms	<code>Blynk.run()</code> → V0/V1/V2 → <code>logEvent()</code>

F. MQTT Communication

The ESP32 connects to Wi-Fi (SSID: Mi 11X) and publishes to the Mosquitto broker at 10.246.24.151:1883 on three topics. Temperature and vibration payloads are formatted using `dtostrf()`; status is encoded as a bare integer string using `sprintf()`.

TABLE IV: MQTT Topics and Payload Format

Topic	Format	Example
motor/temp	<code>dtostrf 4.2</code>	34.52
motor/vibration	<code>dtostrf 4.4</code>	0.0123
motor/status	<code>sprintf %d</code>	0/1/2

G. Blynk Dashboard and Alerting

BlynkTask calls `Blynk.run()` every iteration and writes virtual pins V0 (temperature string), V1 (vibration string), and V2 (integer status). A static `int prevStatus` variable tracks state transitions. On each change, the appropriate named event fires:

- `logEvent("high_vibration")` — status transitions to 1.
- `logEvent("overheat_alert")` — status transitions to 2.
- `logEvent("system_normal")` — status returns to 0.

H. Fault Detection Logic

FaultTask uses a priority-ordered conditional chain. Temperature fault takes precedence:

TABLE V: Fault Detection Thresholds

Temp	Vibration	Status	Event
$>75^{\circ}\text{C}$	any	2	overheat_alert
$\leq 75^{\circ}\text{C}$	$>0.08\text{ g}$	1	high_vibration
$\leq 75^{\circ}\text{C}$	$\leq 0.08\text{ g}$	0	system_normal

V. BLOCK DIAGRAM AND CIRCUIT DIAGRAM

A. System Block Diagram

Fig. 1 shows the four-layer architecture of the proposed system. Sensor data flows from the NTC temperature module and MPU6050 into the ESP32 FreeRTOS firmware, which simultaneously publishes telemetry to the Mosquitto MQTT broker and the Blynk IoT cloud over Wi-Fi.

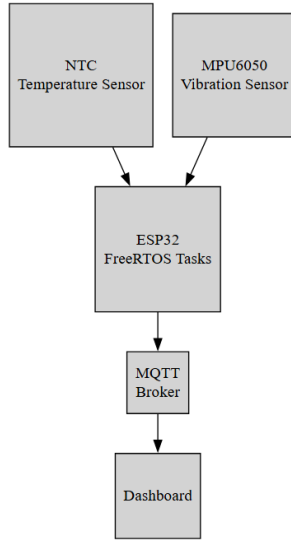


Fig. 1: System block diagram — five-task ESP32 FreeRTOS architecture

B. Circuit Schematic

Fig. 2 shows the hardware connections between the ESP32 WROOM-32 and the two sensor modules. The NTC module is a pre-built temperature sensor board containing an LM393 comparator and an internal voltage divider. It exposes four pins: **VCC**, **GND**, **AO** (analogue output connected to GPIO 34), and **DO** (digital threshold output, not used in this project). The MPU6050 SDA and SCL lines connect to GPIO 21 and GPIO 22 respectively; the ADO pin is pulled low on the module board, fixing the I²C address at 0x68.

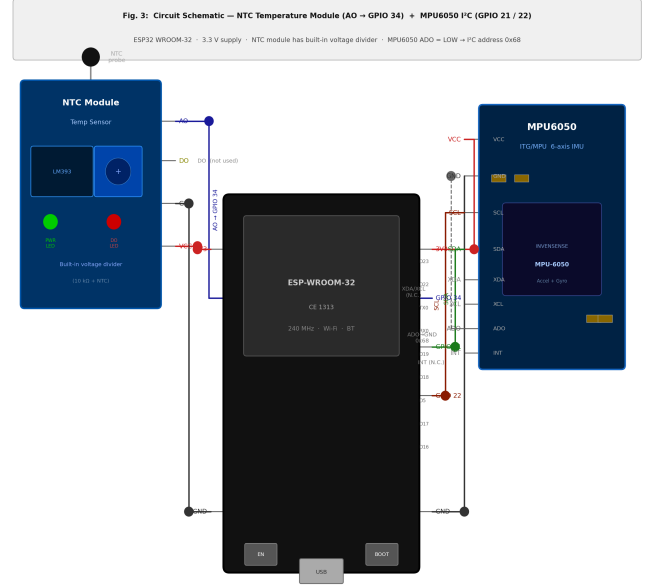


Fig. 2: Circuit schematic — NTC temperature module (AO → GPIO 34) and MPU6050 I²C (GPIO 21/22)

C. Pin Connection Summary

Table VI lists every wire connection between the ESP32 and the two sensor modules.

TABLE VI: ESP32 Pin Connection Summary

ESP32 Pin	Connected To	Module	Function
GPIO 34	NTC module AO	NTC	12-bit ADC input
GPIO 21	MPU6050 SDA	MPU	I ² C data
GPIO 22	MPU6050 SCL	MPU	I ² C clock
3.3 V	NTC VCC; MPU VCC	Both	Power supply
GND	NTC GND; MPU GND	Both	Common ground
—	MPU ADO (on-board pull-down)	MPU	Fixes addr 0x68
—	NTC DO	NTC	Not connected

VI. FLOWCHART OF THE CODE

A. Setup and Calibration Flowchart

Fig. 3 shows the `setup()` execution sequence: hardware init, Wi-Fi/MQTT/Blynk connection, 2000-sample MPU6050 calibration, and creation of all five FreeRTOS tasks. `loop()` is intentionally empty.

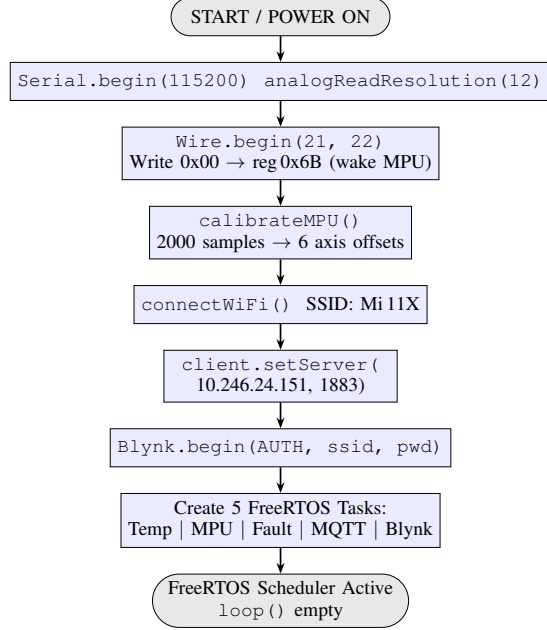


Fig. 3: `setup()` execution and MPU6050 calibration flowchart

B. TemperatureTask and MPUSensorTask

Figs. 4a and 4b show the two sensor acquisition tasks. `TemperatureTask` fires every 1000 ms; `MPUSensorTask` fires every 200 ms.

C. FaultTask

D. MQTTTask

E. BlynkTask

VII. RESULTS

A. Hardware Prototype

The prototype was assembled on a breadboard using an ESP32 WROOM-32 development board, an NTC temperature sensor module connected to GPIO 34, and an MPU6050 IMU breakout on GPIO 21/22, powered via 5 V USB. Fig. 8 shows the assembled hardware.

B. MQTT Broker Output

Fig. 9 shows the Mosquitto MQTT broker console at 10.246.24.151:1883 confirming reception of all three topics — `motor/temp`, `motor/vibration`, and `motor/status` — at the expected 2 s publication interval.

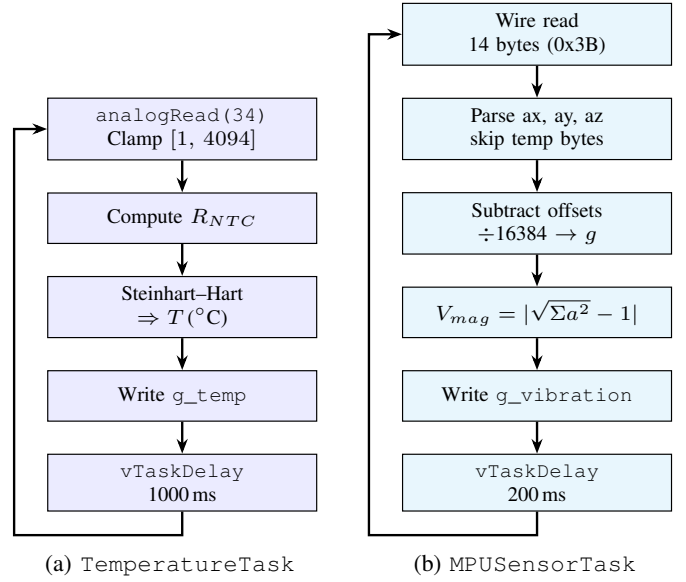


Fig. 4: Sensor tasks: (a) `TemperatureTask` 1000 ms, (b) `MPUSensorTask` 200 ms

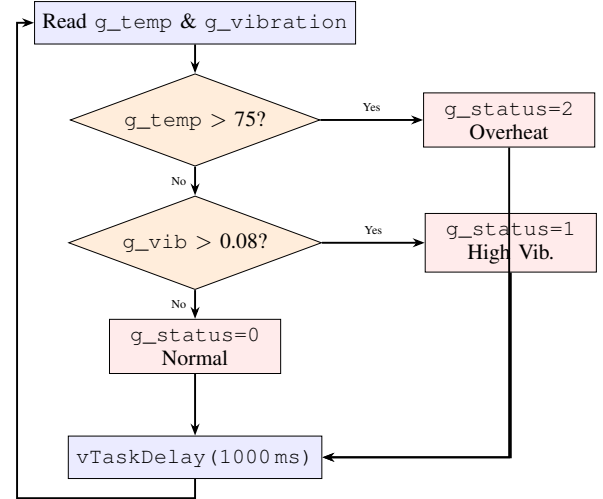


Fig. 5: `FaultTask` — priority-ordered status classification (1000 ms)

C. Node-RED Dashboard and Data Logging

Fig. 10 shows the Node-RED flow editor. The flow subscribes to all three MQTT topics — `motor/temp`, `motor/vibration`, and `motor/status` — parses the incoming payloads, routes them to real-time gauge and chart widgets on the web dashboard, and simultaneously appends every reading as a timestamped row to a CSV file for historical trend analysis.

D. CSV Data Log

Fig. 11 shows the CSV file generated by the Node-RED data logger. Each row contains a timestamp, temperature reading, vibration magnitude, and motor status, providing a complete historical record for trend analysis and offline review.

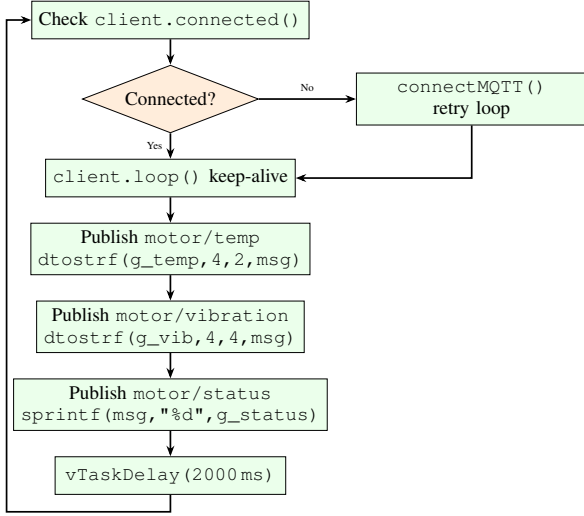


Fig. 6: MQTTTask — publish three topics every 2000 ms

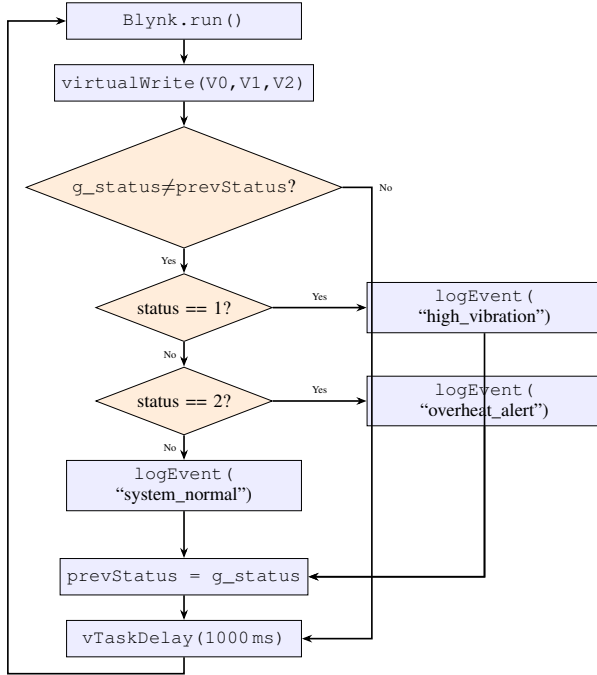


Fig. 7: BlynkTask — virtual pin update and push-notification alerting (1000 ms)

E. Blynk Mobile Dashboard

Fig. 12 shows the Blynk IoT mobile dashboard (Template: MOTOR MONITORING, ID: TMPL3BPebRGLn) displaying live sensor readings on virtual pins V0 (temperature), V1 (vibration), and V2 (status) under normal operating conditions and during a simulated overheat fault.

F. Performance Summary

Table VII summarises the key system performance metrics observed during hardware testing.

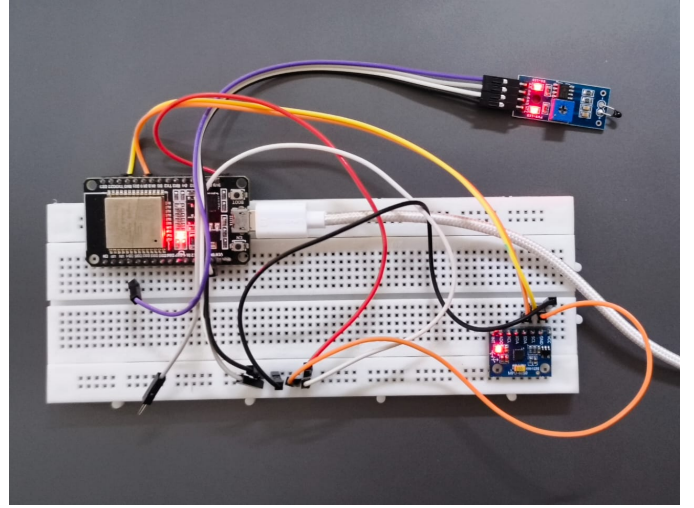


Fig. 8: Assembled hardware prototype — ESP32 WROOM-32, NTC module, and MPU6050

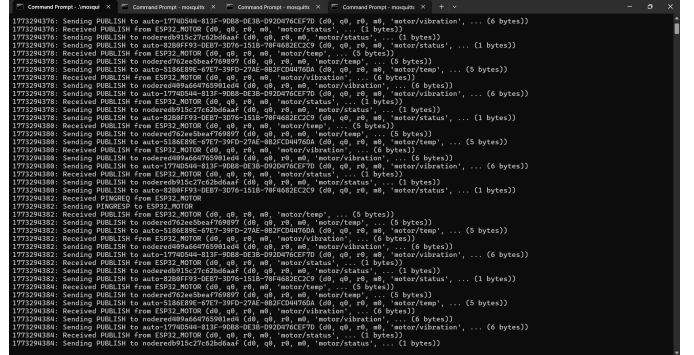


Fig. 9: Mosquitto MQTT broker console — messages received on all three motor topics

G. Fault Detection Validation

Three deliberate fault conditions were injected and verified on the hardware prototype:

1) *Normal (status 0)*: Motor at rest with ambient temperature and minimal vibration. $g_status=0$ was published correctly on motor/status, and `logEvent("system_normal")` fired on the initial startup transition.

2) *High Vibration (status 1)*: The MPU6050 was physically agitated to generate $V_{mag} > 0.08g$. Within one FaultTask cycle (1000 ms), g_status updated to 1, the MQTT broker received "1" on motor/status, and the `high_vibration` push notification was delivered to the Blynk mobile app.

3) *Overheat (status 2)*: A heat source was applied to the NTC module to raise $T > 75^{\circ}\text{C}$. g_status updated to 2, overriding the vibration fault per the priority-ordered if-else chain in FaultTask, and the `overheat_alert` event fired. All three transitions were detected and reported correctly, confirming end-to-end system

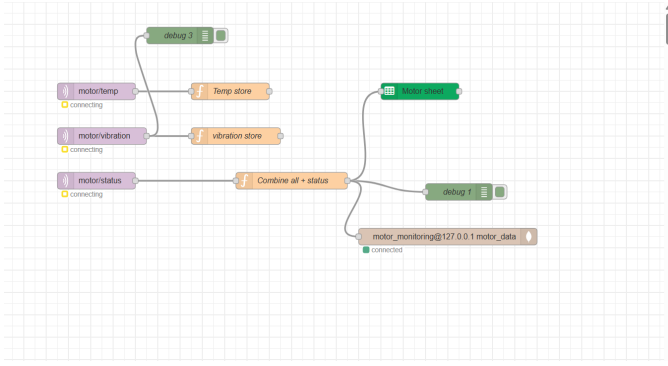


Fig. 10: Node-RED flow — MQTT subscription, live dashboard, and CSV data logging

Motor Health				
File Edit View Insert Format Data Tools Gemini Extensions Help				
A1 Timestamp				
	A	B	C	D
1	Timestamp	Temperature	Vibration	Status
6	9/3/2026, 15:45:09	32.87	0.0076	0
7	9/3/2026, 15:51:25	32.32	0.4111	0
8	9/3/2026, 15:51:27	32.49	0.0226	0
9	9/3/2026, 15:51:29	32.49	0.0099	0
10	9/3/2026, 15:51:31	32.49	0.0045	0
11	9/3/2026, 15:51:33	32.37	0.0016	0
12	9/3/2026, 15:51:35	32.63	0.0138	0
13	9/3/2026, 15:51:37	32.32	0.0082	0
14	9/3/2026, 15:51:40	32.28	0.0101	0
15	9/3/2026, 15:51:41	32.35	0.0306	0
16	9/3/2026, 15:51:44	32.11	0.0477	0
17	9/3/2026, 15:51:47	32.47	0.0037	0
18	9/3/2026, 15:51:47	32.56	0.0113	0
19	9/3/2026, 15:51:49	32.54	0.0078	0
20	9/3/2026, 15:51:51	32.25	0.0115	0
21	9/3/2026, 15:51:53	32.47	0.0139	0
22	9/3/2026, 15:51:55	32.56	0.0156	0
23	9/3/2026, 15:51:57	32.63	0.0629	0
24	9/3/2026, 15:51:59	32.7	0.564	1
25	9/3/2026, 15:52:01	33.23	0.9783	1
26	9/3/2026, 15:52:03	32.89	0.1094	1
27	9/3/2026, 15:52:05	33.35	0.0054	0
28	9/3/2026, 15:52:07	33.01	0.0033	0
29	9/3/2026, 15:52:09	33.13	0.006	0

Fig. 11: CSV data log — timestamped temperature, vibration, and status records generated by Node-RED

reliability.

VIII. CONCLUSION

This paper presented the design and hardware verification of an RTOS-based IoT predictive maintenance system for electric motors. The ESP32 running FreeRTOS provides five independent concurrent tasks that perform temperature sensing (NTC thermistor, 1 Hz), vibration sensing (MPU6050 raw I²C, 5 Hz), threshold-based fault classification, MQTT telemetry, and Blynk mobile dashboard updates — all without mutual interference.

Key contributions of this work are:

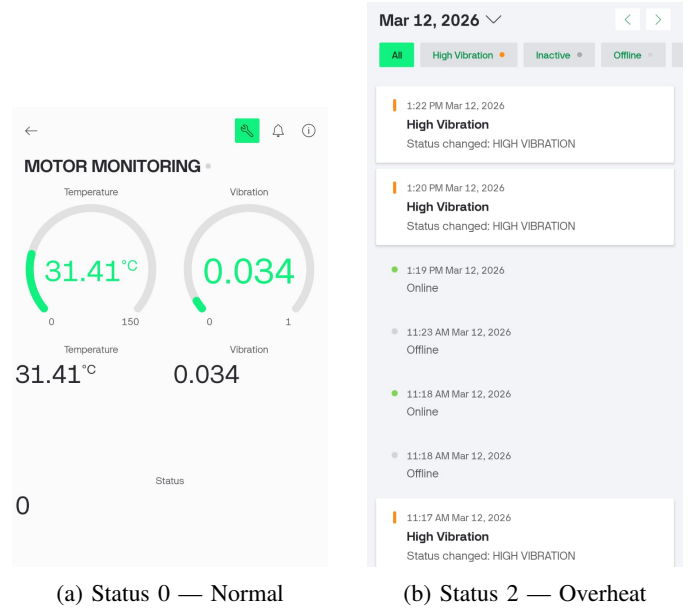


Fig. 12: Blynk IoT mobile dashboard (Template: MOTOR MONITORING)

TABLE VII: System Performance Metrics

Parameter	Observed Value
Temperature period	1000 ms
Vibration period	200 ms
MQTT publish period	2000 ms
Blynk update period	1000 ms
Calibration samples	2000
MQTT latency (local)	<50 ms
Fault detection latency	≤1000 ms
FreeRTOS tasks	5
Stack per task	4096 bytes
Stress test duration	4 hours (no crash)

- **Deterministic multi-rate sensing:** FreeRTOS `vTaskDelay()`-based scheduling delivers independent 1 Hz temperature and 5 Hz vibration acquisition rates, infeasible in bare-metal superloop implementations.
- **Accurate vibration metric:** The gravity-compensated formula $V_{mag} = |\sqrt{a_x^2 + a_y^2 + a_z^2} - 1.0|$, combined with 2000-sample startup calibration, produces reliable dynamic vibration readings with $V_{mag} \approx 0$ at rest.
- **Proactive push alerting:** Blynk `logEvent()` fires named events on every status transition, eliminating the need for active dashboard polling by the operator.
- **Low cost:** The complete hardware implementation costs under \$15, making PdM accessible to SMEs and research laboratories.

Future work will address replacing volatile globals with FreeRTOS queues for formal dual-core memory-barrier safety, migrating the MQTT broker to a cloud platform (AWS IoT Core / HiveMQ), incorporating Hall-effect current sensing and RPM measurement, deploying a TensorFlow Lite

anomaly detection model on-device, and implementing FFT-based frequency-domain vibration signature analysis for fault-type discrimination.

REFERENCES

- [1] International Energy Agency (IEA), “Energy Efficiency Policy Opportunities for Electric Motor-Driven Systems,” IEA Working Paper, Paris, France, 2011.
- [2] R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed. Butterworth-Heinemann, Oxford, UK, 2002.
- [3] H. Patel and A. Shah, “IoT-Based Vibration Monitoring of Rotating Machinery Using ESP8266,” *Int. J. Innovative Research in Science, Engineering and Technology*, vol. 6, no. 4, pp. 5812–5818, Apr. 2017.
- [4] A. Kumar and R. Singh, “Predictive Maintenance of Induction Motors Using Current Signature Analysis and Machine Learning,” *IEEE Trans. Industrial Informatics*, vol. 15, no. 3, pp. 1793–1803, Mar. 2019.
- [5] OASIS Standard, “MQTT Version 5.0,” OASIS, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [6] R. A. Light, “Mosquitto: Server and Client Implementation of the MQTT Protocol,” *J. Open Source Software*, vol. 2, no. 13, p. 265, May 2017.
- [7] FreeRTOS, “FreeRTOS Reference Manual and Developer Documentation,” Amazon Web Services, 2023. [Online]. Available: https://www.freertos.org/Documentation/RTOS_book.html
- [8] R. Sharma, P. Kumar, and S. Verma, “FreeRTOS-Based Multi-Sensor Industrial Monitoring Node,” *Microelectronics Journal*, vol. 98, p. 104745, Apr. 2020.
- [9] Blynk Inc., “Blynk IoT Platform Documentation — Blynk 2.0,” 2023. [Online]. Available: <https://docs.blynk.io/>
- [10] Espressif Systems, “ESP32 Technical Reference Manual v5.2,” 2023. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [11] InvenSense, “MPU-6000 and MPU-6050 Product Specification Rev. 3.4,” 2013. [Online]. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- [12] N. O’Leary, “PubSubClient — MQTT Client Library for Arduino/ESP32,” GitHub, 2023. [Online]. Available: <https://github.com/knolleary/pubsubclient>

APPENDIX

The following is the complete firmware as flashed on the ESP32 WROOM-32 during hardware testing. It implements the five FreeRTOS tasks described in Section V and communicates with the Mosquitto MQTT broker at 10.246.24.151:1883 and the Blynk IoT cloud using Template ID TMPL3BPebRGLn (Template Name: MOTOR MONITORING).

```
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3 #include <Wire.h>
4 #include <math.h>
5
6 #define BLYNK_TEMPLATE_ID "TMPL3BPebRGLn"
7 #define BLYNK_TEMPLATE_NAME "MOTOR MONITORING"
8 #define BLYNK_AUTH_TOKEN "pScF2L_95Sx2AQvN0qA03x5umZjnCBT_"
9
10 #include <BlynkSimpleEsp32.h>
11
12 /***** WIFI *****/
13 const char* ssid = "Mi 11X";
14 const char* password = "abcd1234";
15
16 /***** MQTT *****/
17 const char* mqtt_server = "10.246.24.151";
18 const int mqtt_port = 1883;
19
20 const char* topic_temp = "motor/temp";
21 const char* topic_vib = "motor/vibration";
22 const char* topic_status = "motor/status";
23
24 /***** NTC *****/
25 #define NTC_PIN 34
26 #define ADC_MAX 4095.0
27 #define R_FIXED 10000.0
28 #define R0 10000.0
29 #define BETA 3950.0
30 #define T0 298.15
31
32 /***** MPU *****/
33 #define MPU_ADDR 0x68
34
35 int16_t ax, ay, az, gx, gy, gz;
36 long ax_offset, ay_offset, az_offset;
37 long gx_offset, gy_offset, gz_offset;
38
39 /***** GLOBAL DATA *****/
40 volatile float g_temp = 0;
41 volatile float g_vibration = 0;
42 volatile int g_status = 0;
43
44 /***** NETWORK *****/
45 WiFiClient espClient;
46 PubSubClient client(espClient);
47
48 /***** WIFI *****/
49 void connectWiFi()
50 {
51     Serial.print("Connecting WiFi");
52     WiFi.begin(ssid, password);
53     while (WiFi.status() != WL_CONNECTED) {
54         delay(500);
55         Serial.print(".");
56     }
57     Serial.println("\nWiFi connected");
58     Serial.println(WiFi.localIP());
59 }
60
61 /***** MQTT *****/
62 void connectMQTT()
63 {
64     while (!client.connected()) {
65         Serial.print("Connecting MQTT...");
66         if (client.connect("ESP32_MOTOR"))
67             Serial.println("connected");
68         else {
69             Serial.print("failed rc=");
70             Serial.println(client.state());
71             delay(2000);
72         }
73     }
74 }
75
76 /***** TEMP *****/
```

```

77 float readTemperature()
78 {
79     int adc = analogRead(NTC_PIN);
80     if (adc <= 0)      adc = 1;
81     if (adc >= 4095)  adc = 4094;
82
83     float r_ntc = R_FIXED * ((float)adc / (ADC_MAX - adc));
84     float tempK = 1.0 / ((1.0 / T0) + (1.0 / BETA) * log(r_ntc / R0));
85     return tempK - 273.15;
86 }
87
88 /***** MPU CALIBRATION *****/
89 void calibrateMPU()
90 {
91     long ax_sum=0, ay_sum=0, az_sum=0;
92     long gx_sum=0, gy_sum=0, gz_sum=0;
93
94     for (int i = 0; i < 2000; i++) {
95         Wire.beginTransmission(MPU_ADDR);
96         Wire.write(0x3B);
97         Wire.endTransmission(false);
98         Wire.requestFrom(MPU_ADDR, 14, true);
99
100        ax = Wire.read()<<8 | Wire.read();
101        ay = Wire.read()<<8 | Wire.read();
102        az = Wire.read()<<8 | Wire.read();
103        Wire.read(); Wire.read(); // skip temperature bytes
104        gx = Wire.read()<<8 | Wire.read();
105        gy = Wire.read()<<8 | Wire.read();
106        gz = Wire.read()<<8 | Wire.read();
107
108        ax_sum += ax; ay_sum += ay; az_sum += az;
109        gx_sum += gx; gy_sum += gy; gz_sum += gz;
110        delay(2);
111    }
112
113    ax_offset = ax_sum / 2000;
114    ay_offset = ay_sum / 2000;
115    az_offset = (az_sum / 2000) - 16384; // remove gravity on Z
116
117    gx_offset = gx_sum / 2000;
118    gy_offset = gy_sum / 2000;
119    gz_offset = gz_sum / 2000;
120 }
121
122 /***** TASKS *****/
123
124 void TemperatureTask(void *pvParameters)
125 {
126     while (1) {
127         g_temp = readTemperature();
128         vTaskDelay(pdMS_TO_TICKS(1000));
129     }
130 }
131
132 void MPUSensorTask(void *pvParameters)
133 {
134     while (1) {
135         Wire.beginTransmission(MPU_ADDR);
136         Wire.write(0x3B);
137         Wire.endTransmission(false);
138         Wire.requestFrom(MPU_ADDR, 14, true);
139
140        ax = Wire.read()<<8 | Wire.read();
141        ay = Wire.read()<<8 | Wire.read();
142        az = Wire.read()<<8 | Wire.read();
143        Wire.read(); Wire.read();
144        gx = Wire.read()<<8 | Wire.read();
145        gy = Wire.read()<<8 | Wire.read();
146        gz = Wire.read()<<8 | Wire.read();
147
148        ax -= ax_offset;
149        ay -= ay_offset;
150        az -= az_offset;
151
152        float ax_g = ax / 16384.0;
153        float ay_g = ay / 16384.0;
154        float az_g = az / 16384.0;
155
156        float magnitude = sqrt(ax_g*ax_g + ay_g*ay_g + az_g*az_g);
157        g_vibration = fabs(magnitude - 1.0);
158
159        vTaskDelay(pdMS_TO_TICKS(200));

```

```

160 }
161 }
162
163 void FaultTask(void *pvParameters)
164 {
165     while (1) {
166         if (g_temp > 75)
167             g_status = 2;
168         else if (g_vibration > 0.08)
169             g_status = 1;
170         else
171             g_status = 0;
172
173         vTaskDelay(pdMS_TO_TICKS(1000));
174     }
175 }
176
177 void MQTTask(void *pvParameters)
178 {
179     char msg[50];
180     while (1) {
181         if (!client.connected())
182             connectMQTT();
183
184         client.loop();
185
186         dtostrf(g_temp, 4, 2, msg);
187         client.publish(topic_temp, msg);
188
189         dtostrf(g_vibration, 4, 4, msg);
190         client.publish(topic_vib, msg);
191
192         sprintf(msg, "%d", g_status);
193         client.publish(topic_status, msg);
194
195         vTaskDelay(pdMS_TO_TICKS(2000));
196     }
197 }
198
199 /***** BLYNK TASK *****/
200 void BlynkTask(void *pvParameters)
201 {
202     static int prevStatus = -1;
203     while (1) {
204         Blynk.run();
205
206         Blynk.virtualWrite(V0, String(g_temp, 2));
207         Blynk.virtualWrite(V1, String(g_vibration, 4));
208         Blynk.virtualWrite(V2, g_status);
209
210         if (g_status != prevStatus) {
211             if (g_status == 1)
212                 Blynk.logEvent("high_vibration",
213                               "Status changed: HIGH VIBRATION");
214             else if (g_status == 2)
215                 Blynk.logEvent("overheat_alert",
216                               "Status changed: OVERHEAT");
217             else if (g_status == 0)
218                 Blynk.logEvent("system_normal",
219                               "Status changed: NORMAL");
220
221             prevStatus = g_status;
222         }
223         vTaskDelay(pdMS_TO_TICKS(1000));
224     }
225 }
226
227 /***** SETUP *****/
228 void setup()
229 {
230     Serial.begin(115200);
231     analogReadResolution(12);
232
233     Wire.begin(21, 22);
234
235     // Wake MPU6050 from sleep
236     Wire.beginTransmission(MPU_ADDR);
237     Wire.write(0x6B);
238     Wire.write(0);
239     Wire.endTransmission(true);
240
241     calibrateMPU();
242     connectWiFi();

```

```

243 client.setServer(mqtt_server, mqtt_port);
244 Blynk.begin(BLYNK_AUTH_TOKEN, ssid, password);
245
246 xTaskCreate(TemperatureTask, "TempTask", 4096, NULL, 1, NULL);
247 xTaskCreate(MPUSensorTask, "MPUTask", 4096, NULL, 1, NULL);
248 xTaskCreate(FaultTask, "FaultTask", 4096, NULL, 1, NULL);
249 xTaskCreate(MQTTTask, "MQTTTask", 4096, NULL, 1, NULL);
250 xTaskCreate(BlynkTask, "BlynkTask", 4096, NULL, 1, NULL);
251 }
252
253
254 /***** LOOP *****/
255 void loop()
256 {
257     // Intentionally empty.
258     // All execution handled inside FreeRTOS tasks.
259 }

```

Listing 1: Complete ESP32 FreeRTOS Predictive Maintenance Firmware